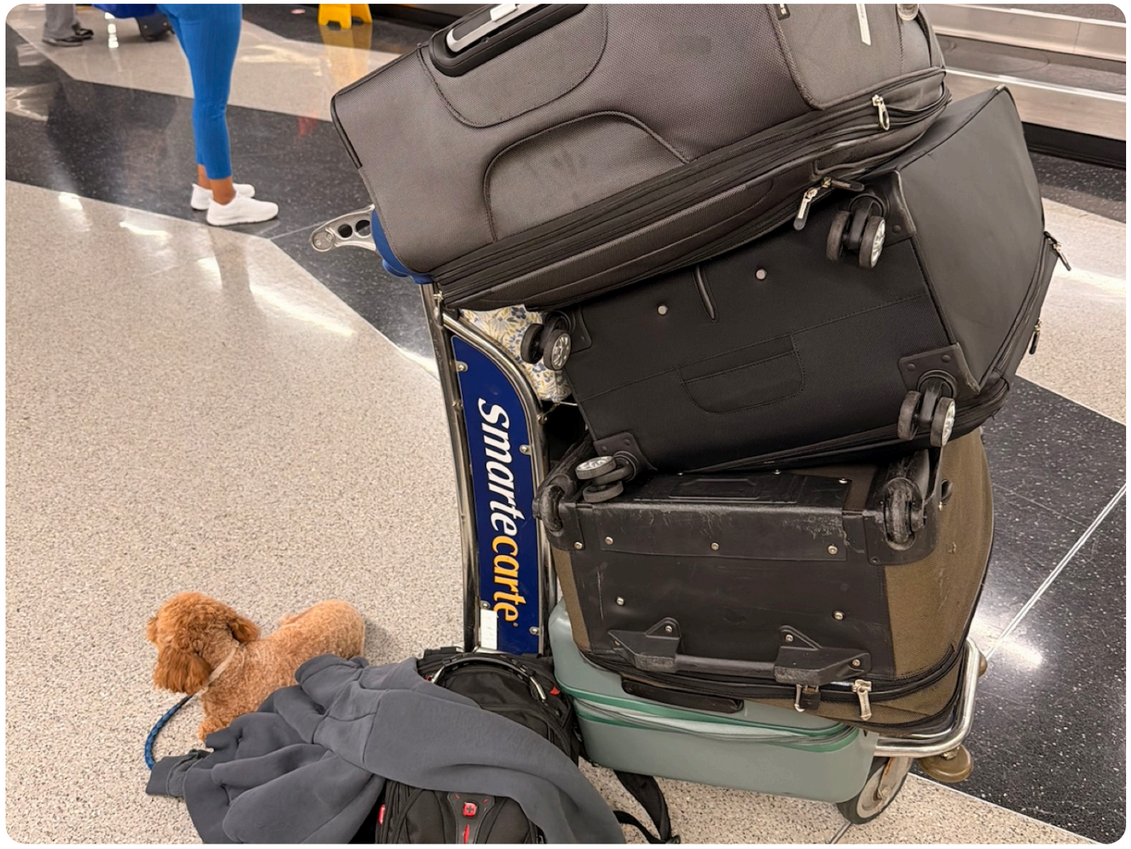
SRE AND OBSERVABILITY

Reliability before the incident.

How developers and SREs catch problems before users do.

Dusan Stanojevic

tracewayapp.com



A talk about lost bags, green boards, and knowing before you fly.

Available is not reliable

The opening story was fully available.

AVAILABLE

It answers.

“Check your inbox” came back in 38 milliseconds.

Uptime checks, status pages and most dashboards measure this.

RELIABLE

It keeps its promise.

The email arrives within 60 seconds.

This is what the person needed.

AT THE BELT

Why we find out late.

You lose the bag at the belt. You find out at baggage claim. Then you write it up.

BAG TAG 3 MIN

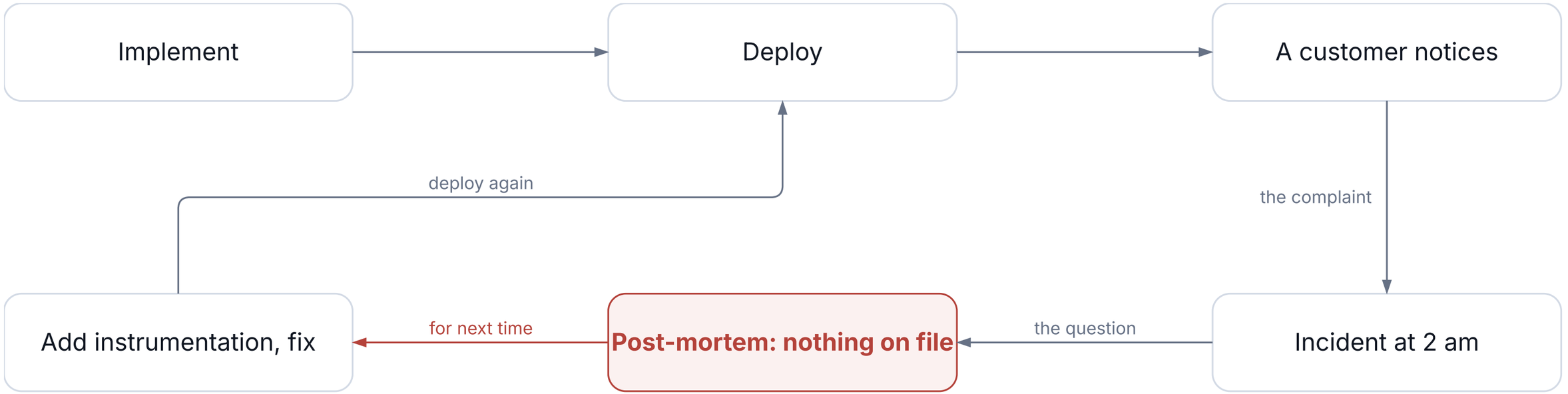
FINDING OUT

- [ ] the default loop
- [ ] reactive and proactive
- [ ] when, not where

FINAL CALL ▸

The default loop

A customer notices. An incident at 2 am. A post-mortem. Then we add the instrumentation we needed yesterday.



Reactive and proactive

Both teams do both. The difference is timing.

	REACTIVE	PROACTIVE
Starts from	A problem in production	A promise we want to keep
Asks	What broke? How do we fix it?	What can break? How will we know?
Produces	A fix and a new dashboard	Checks, tests and a recovery plan

Developers see the code. SREs see the failures. Both need to be in the room before the first deploy, not after the first incident.

THE WHOLE TALK IN ONE LINE

The difference is when we do the work, not where the data comes from.

You find out at baggage claim, not at the gate. The earlier the two teams talk, the earlier the question gets asked.

AT CHECK-IN

Say what the promise is.

Write down what “you'll get an email” means. Then decide what to measure.

BAG TAG 3.5 MIN

THE PROMISE

- [ ] the example
- [ ] the promise
- [ ] four words
- [ ] four kinds of check
- [ ] prevent, detect, call

FINAL CALL ▸

The example: a password reset email

Four steps from a click to an inbox. Every slide from here uses these names.

1

Request

Someone forgets their password and clicks reset.

`POST /password-resets`

2

Accept

The API writes a reset row and answers “Check your inbox”.

`reset: requested`

3

Send

A sender takes the job off the queue and calls the email provider.

`sender, provider`

4

Deliver

The email arrives in the inbox.

`reset: delivered`

Process dies here: row written, job never queued

THE PROMISE

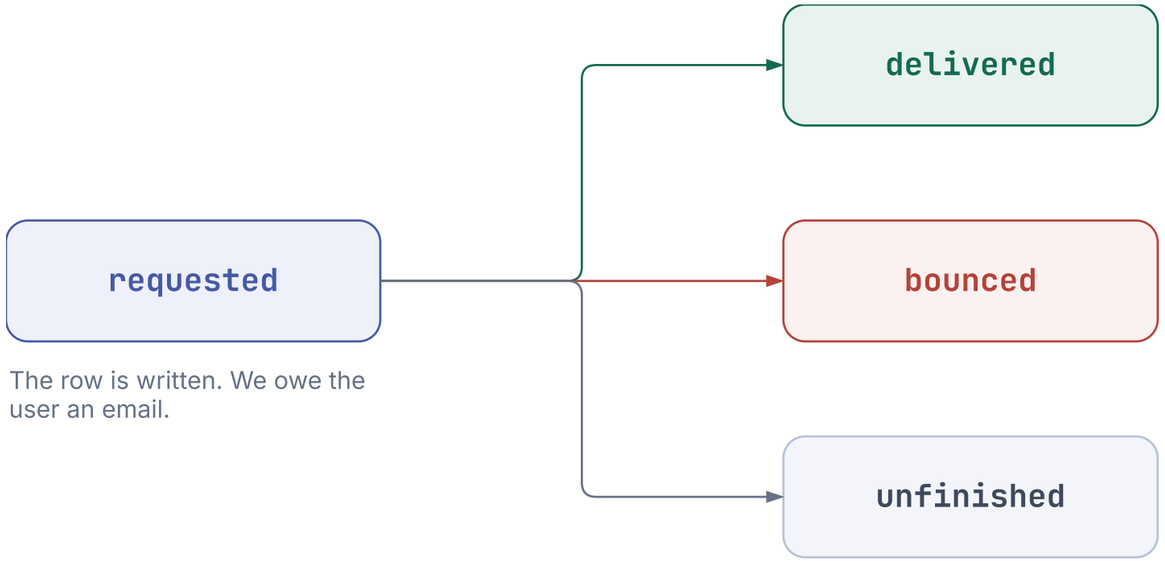
99.9% of password resets are delivered within 60 seconds.

99.9%	OF RESETS
60 s	DEADLINE
30 d	ROLLING WINDOW

“Check your inbox” means we took the request. It does not mean an email exists.

Four words

Requested. Delivered. Bounced. Somewhere in between.



In the inbox, on time.

Bounced, and we know it.

Everything else. Waiting, sending, or we do not know.

```
requested = delivered + bounced + unfinished    same resets, same moment
```

Four kinds of check

Each one answers a different question. Know which before you alert on it.

THE RULE	EXAMPLE	HOW WE CHECK
Never happens	Two resets for one request	A unique key on the request
Never stuck	A requested reset nobody will ever send	Count requested rows that have not moved in 20 seconds
On time	99.9% delivered within 60 s	Judge every reset at its deadline, finished or not
Early warning	The oldest unsent reset is getting old	Age and backlog, with a plan for what to do

No bag flies without its passenger. No reset sits without a sender.

Prevent, detect, call

Three jobs. Trouble starts when one tool is asked to do all three.

PREVENT

A constraint

Makes the wrong thing impossible.

The tag printer.

what slipped →

DETECT

A reconciliation query

Finds what slipped through.

The bag count before pushback.

what it found →

CALL

An alert

Gets it to someone who can act.

The radio call to the ramp.

EVERY HANDOFF

Make it observable.

If you can tell where every bag is from the scans alone, the system is observable. Scan at every handoff. Trust the records, not memory.

BAG TAG 5 MIN

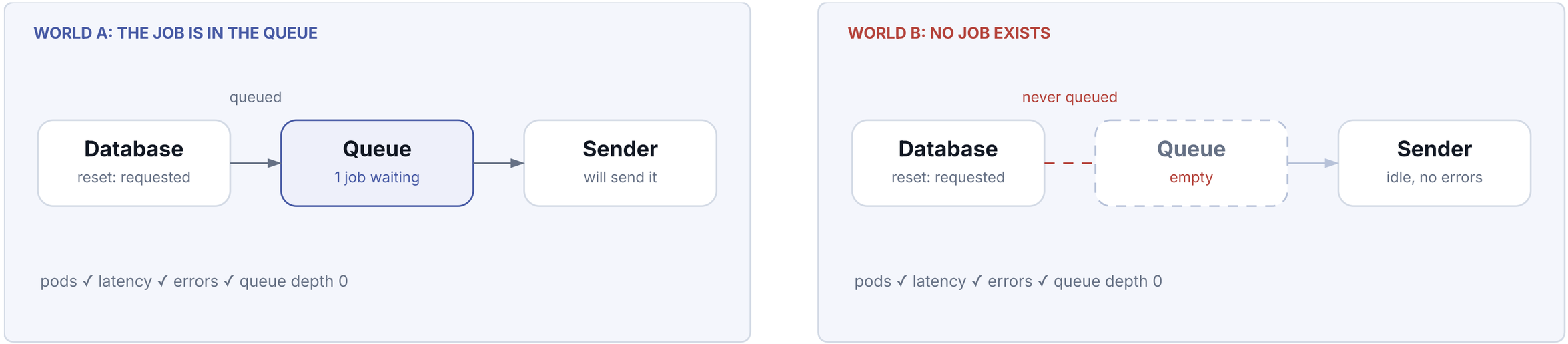
THE STATE

- [ ] two worlds
- [ ] what we need to know
- [ ] one row
- [ ] the checker
- [ ] the metrics
- [ ] one scan

FINAL CALL ▸

Two worlds, one dashboard

The job is in the queue, or no job exists. The boards look the same.



If both worlds look the same on the dashboard, the dashboard is decoration.

What we need to know

Where is every reset right now, and how do we know?

THE QUESTION	WHAT ANSWERS IT
What did we promise?	A reset row, and when we took the request
Where is it now?	Its state, and when it last moved
Is it moving?	The oldest unfinished reset, and how many were delivered on time
Why is it waiting?	A reason code, the provider, the next retry
What changed?	Service version and config revision

One row

The row is the promise and the job. Written before “Check your inbox”, read by the sender from the table.

```
# the API
reset = find_or_create(request_key, user_id, state="requested")
return 200 "Check your inbox"

# the sender, in a loop
for reset in requested_resets():
    outcome = provider.send(reset)
    record(reset, outcome)          # delivered or bounced
```

- **Same key, same reset**
  Click twice, get the same reset back.
- **Durable before the reply**
  The row exists before anyone hears “Check your inbox”.
- **The sender reads the table**
  No queue between the row and the job, so nothing to lose.
- **What the provider said**
  Delivered or bounced. Nothing in between counts as done.

At scale, the transactional outbox is the same idea with a table of its own.

The checker

Count the resets before anyone complains. Every requested reset with no progress in 20 seconds.

```sql
SELECT r.state                                          AS stage,
       count(*)                                         AS unfinished,
       max(extract(epoch FROM now() - r.requested_at)) AS oldest_age_s
FROM   password_resets r
WHERE  r.state NOT IN ('delivered', 'bounced')
  AND  r.requested_at < now() - interval '20 seconds'
GROUP  BY r.state;
```

- **Unfinished, per stage**
  The number on the dashboard.
- **The oldest one**
  How long it has waited.
- **What counts as unfinished**
  Anything not delivered or bounced.
- **Grace period**
  Keeps healthy in-flight resets out.

This query finds world B. A log line from a dead process never will.

The metrics

How many per stage, how old the oldest is, and whether the count is fresh.

METRIC	MEANING
`password_reset_unfinished{stage}`	Unfinished resets in each stage
`password_reset_oldest_unfinished_age_seconds{stage}`	Age of the oldest unfinished reset
`password_reset_deadline_results_total{result}`	Resets judged at their deadline, met or missed
`password_reset_checker_last_success_timestamp_seconds`	When the checker last finished a full scan

Keep labels small. Reset ids go in the table, not in metrics.

One scan

Who had the reset, when, and why it waited.



- **The reset's identity**
  Same across every retry.
- **Still requested**
  A provider timeout is not a delivery, so the state does not move.
- **A short reason**
  Small enough to be a label.
- **The trace**
  Links this scan to the rest of the journey.

SNOW DAY

Break it before release.

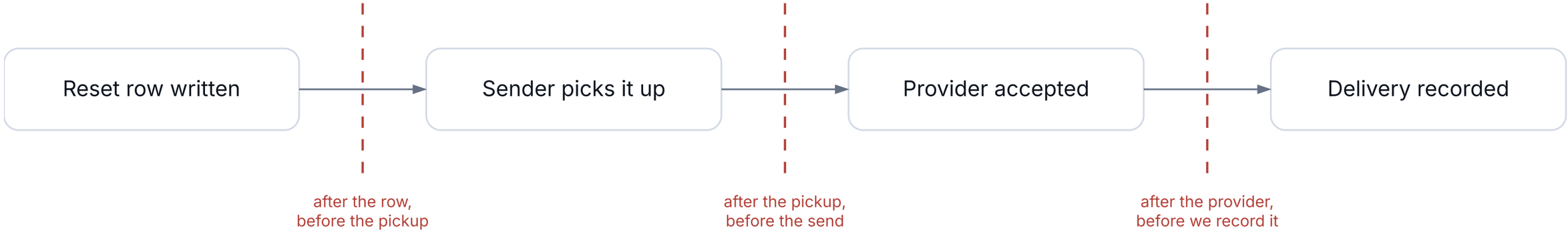
Ground crews practice the snow day before it snows. So should we.



SNOW-DAY DRILL, EVERY BAG ACCOUNTED FOR

Where to cut

Row, pickup, send, record. Every handoff is a place to lose an email.



Lose twenty resets on purpose

Kill the sender after it picks up a reset, before the send, on 2% of resets. The checker must find all twenty. A new sender must deliver them.

1

2%

A named stop

Test builds exit at a named barrier: after the pickup, before the send.

→

2

20 stuck

A known number

1,000 resets go in. Twenty hit the barrier and stay requested.

→

3

20 found

The checker sees it

One scan reports exactly twenty unfinished, no more and no fewer.

→

4

0 left

Recovery finishes it

A fresh sender drains them. Count the emails in a test mailbox.

Pass: 1,000 delivered, none unfinished after 30 seconds, no duplicate emails.

The failure matrix

Walk the first row. The last two rows are the snow day.

WHAT WE BREAK	WHAT SHOULD HAPPEN	WHAT WE CHECK
Sender dies after picking up a reset, before the send	The reset stays requested; nothing is lost	The checker sees it; a new sender finishes it
One sender stalls, others fine	Resets get old but stay recoverable	Age shows it; another sender takes over
Provider slows down	Retries stay within limits	We see the backlog and stop retrying blindly
The checker stops	Resets are fine; our view is not	Stale is reported as stale, not green

Three things every test checks

Most teams test the first one. The lost email hid in the other two.

BEHAVIOR

Did it hold up?

Did the rule hold, or did we land in the degraded state we planned for?

Count emails in a test mailbox, not in the service's own metric.

EVIDENCE

Could we see it?

Could someone watching tell this from a healthy day?

Set one reset back to requested on purpose and make sure the checker reports exactly one.

RESPONSE

Did we recover?

Did a retry, a new sender or a person put things right?

A new sender finishes the twenty with no duplicate emails.

THE RADIO

Tighten the loop.

Every failure is found by someone. Make sure it is not the customer.

BAG TAG 6 MIN

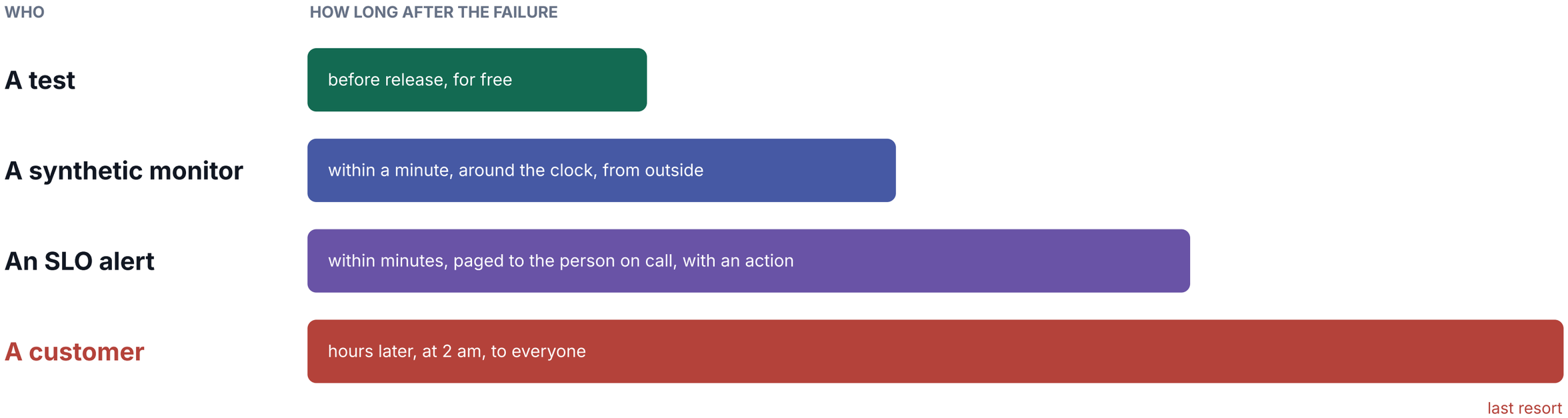
WHO FINDS OUT

- [ ] the ladder
- [ ] known promises
- [ ] monitors
- [ ] exception tracking
- [ ] the burn-rate alert
- [ ] on call: is there time?
- [ ] three ways out

FINAL CALL ▸

Who finds out first

Every failure is found by someone. The question is who, and how long after.



The default today is the bottom rung. Every tool in this section moves a kind of failure up the ladder.

Some promises are already written

Every endpoint gets latency bands, an Apdex and an impact score before anyone writes a rule. The reset email gets nothing until the app emits its own state.

Endpoint	Calls	Typical	Slow	Awful	Impact
POST /api/checkout	250	398ms	931ms	2.7s	Good
GET /api/recommendations	246	390ms	761ms	1.6s	Good
GET /api/search	322	236ms	711ms	1.6s	Good
GET /api/products/:id	703	38ms	100ms	206ms	Good

TRACEWAY ENDPOINTS, DEMO PROJECT

TYPICAL, SLOW, AWFUL

P50, P95 and P99 per route, from the first request.

APDEX

Satisfied up to 750 ms, tolerating up to 1.5 s. Slower, or a 5xx, is frustrated.

IMPACT

The worst of Apdex, 5xx rate, P99, 4xx rate and error volume. Medium 0.25, high 0.50, critical 0.75.

Monitors

Request a reset every minute and see if the email comes out the other end.

1. **Uptime check**

   `GET /health` every 30 seconds. Proves it answers.

2. **Journey check**

   Request a reset for a mailbox you control, poll it, fail after 60 seconds. Proves it keeps the promise.

This would have caught the opening story within minutes, from outside, with no change to the service.

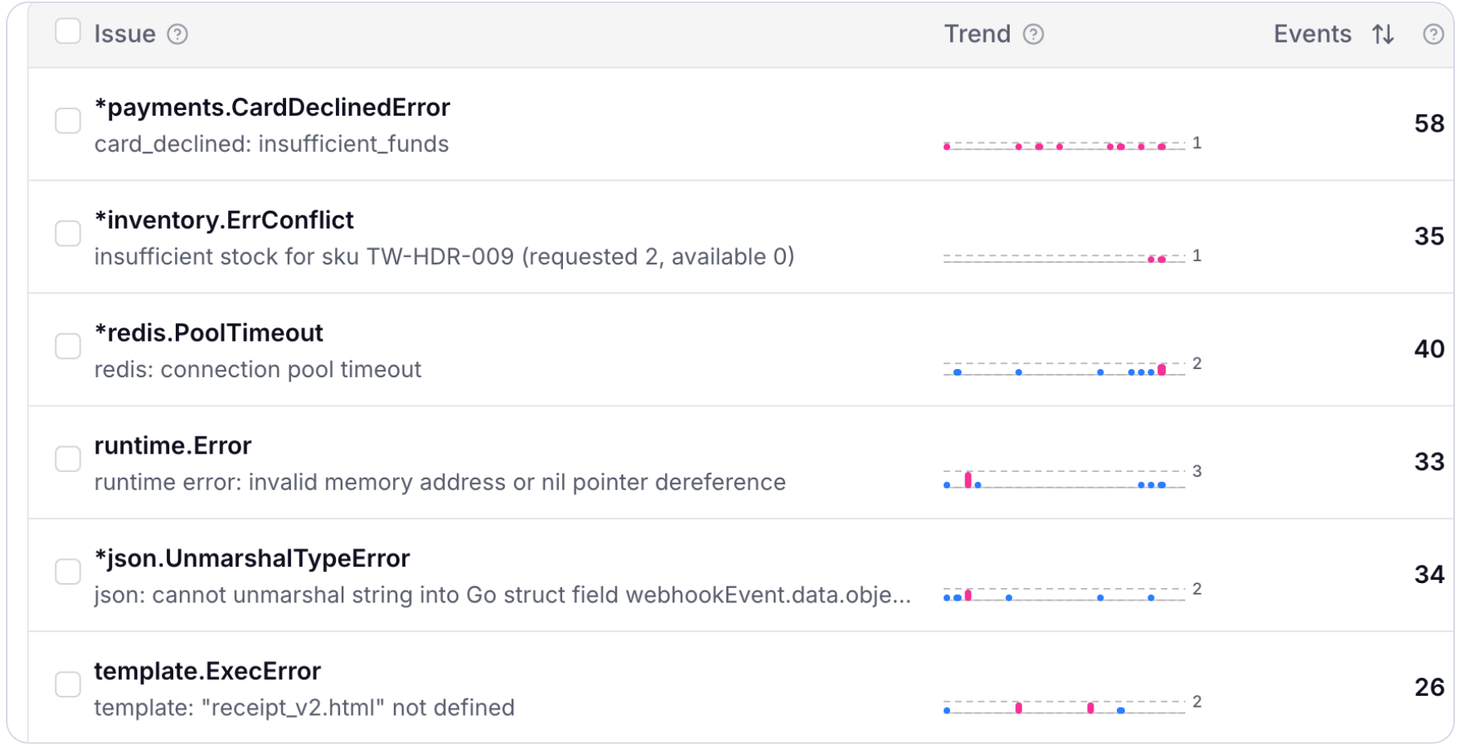
Flag synthetic resets and keep them out of the SLO. A monitor covers the path you script. The checker covers every reset.



STATUS PAGE FROM UPTIME CHECKS. TRACEWAY, DEMO DATA

Exception tracking

Every failure that throws, grouped by cause and linked to the reset.



TRACEWAY ISSUES, DEMO PROJECT

- **One issue, not twenty alerts**
  Grouped by cause, with a count and a first-seen time.
- **Linked to the reset**
  One click from the error to the reset, its attempts and its trace.
- **What it cannot see**
  An email that simply never went throws nothing. That is the checker's job, and the monitor's.

The burn-rate alert

This is the page. The standard two-window alert, with two details that are ours.

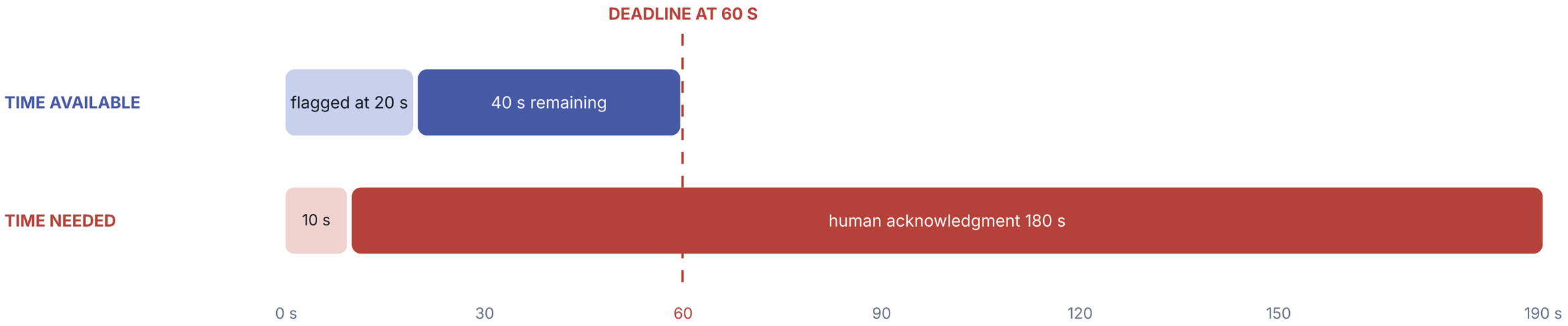
```
- alert: PasswordResetFastBurn
  expr: |
    ( sum(rate(password_reset_deadline_results_total{result="missed"}[1h]))
      / sum(rate(password_reset_deadline_results_total[1h])) ) > 14.4 * 0.001
    and
    ( sum(rate(password_reset_deadline_results_total{result="missed"}[5m]))
      / sum(rate(password_reset_deadline_results_total[5m])) ) > 14.4 * 0.001
  for: 2m
  labels: { severity: page }
```

- **One hour**
  Is the budget really burning?
- **Five minutes**
  Is it still burning now?
- **The threshold**
  Both windows must cross it.

The counter comes from the table, so restarts do not reset it. Each reset is judged at its deadline, so the alert runs a minute behind.

On call: is there time?

The reset expires in 40 seconds. The on-call phone rings in 190.



40 seconds available, 190 needed. A page cannot save this reset. The schedule still matters: someone has to own the next one, and a page without an action is just a notification.

Three ways out

If the math does not work, change the plan, not the threshold.

AUTOMATE

A second sender sweeps the table

It finishes what the first one left. No human needed for the common case.

BE HONEST

Change what the page says

“Delayed, try again in a minute”, and offer another way in. Protect the resets already in flight.

PROMISE LESS

Promise what you can keep

If 60 seconds is not realistic, say five minutes, and mean it.



Carry-on only. Give people a way in that does not depend on the email arriving.

THE TAG

One contract, both teams.

Developers know where the bag changes hands. SREs know how it gets lost. Write it down together, before the first flight.

BAG TAG 3.5 MIN

TOGETHER

- [ ] the contract
- [ ] walk one bag through
- [ ] the loop we want

FINAL CALL ▸

The contract

Printed on the tag. Agreed by developers and SREs before the first email is sent.

WORKFLOW	request and deliver a password reset email
PROMISE	delivered within 60 s, 99.9% of the time
RULES	one email per request; nothing gets stuck
WE ASSUME	the database is durable; the provider's answer is the truth
WE WATCH	state, age, deadline results, checker freshness
WE BREAK	crash points, stalled senders, a slow provider, a stopped checker
WE RECOVER	a new sender finishes what was left; change the page when we are slow
WE OWN IT	one team, one on-call escalation path

Walk one reset through the code

A developer and an SRE, one reset, from the click to the inbox.

Where is it durable?

If it lives in memory, it dies with the process.

Who moves it next?

A row nobody polls is a lost email with paperwork.

What if that owner disappears?

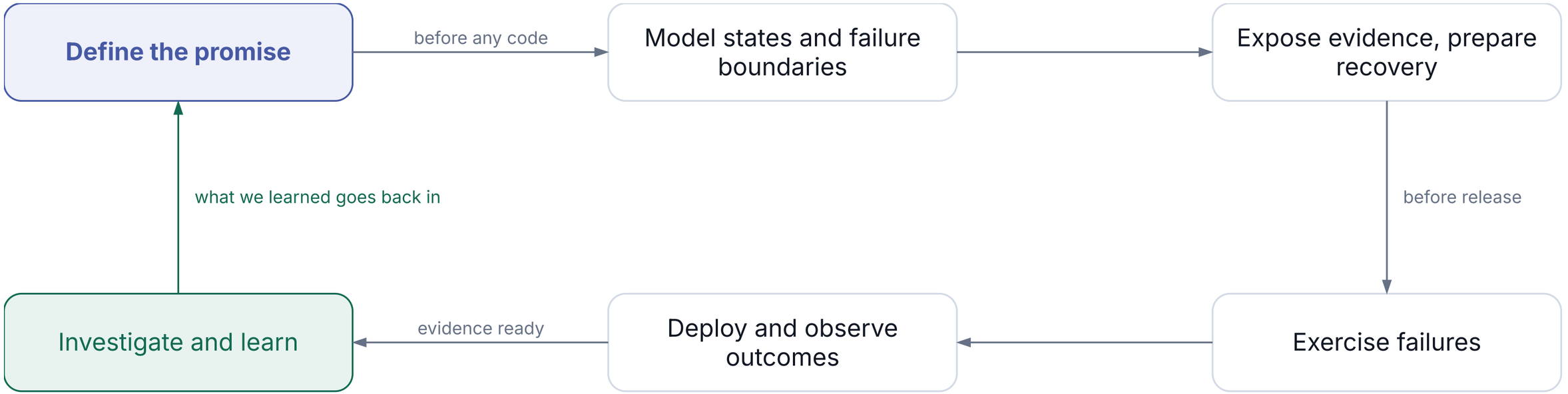
That handoff is a boundary. Test it.

How would we know it is lost, not late?

If nothing tells you, you are in world B.

The loop we want

SREs in the room before the first commit, not after the first incident.



FINAL CALL

Last resort, not default.

A customer at 2 am should be the last to find out, not the first. Pick one promise. Make it visible. Break it before release. Do it together.

Dusan Stanojevic

tracewayapp.com



Before we ship, we should be able to say how we will know it is working, and who finds out first when it is not.